

Control Statements

Introduction

Control Statements are used to control the flow of execution of a program.

Normally, Python executes statements one after another from top to bottom. Control statements allow the program to make decisions, repeat tasks, or alter the normal flow of execution based on specific conditions.

Control statements help the program decide:

- Which code should execute
- When code should execute
- How many times code should execute
- Whether execution should continue, stop, or skip

Types of Control Statements

Python control statements are broadly classified into three categories:

Type	Purpose
Conditional Statements	Execute code based on conditions
Looping Statements	Execute code repeatedly
Jump Statements	Alter the normal flow of loops

Why Control Statements Are Important

Control statements help programs:

- Make decisions
- Repeat tasks efficiently
- Avoid unnecessary code repetition
- Handle different situations dynamically
- Control program execution flow

Without control statements, programs would execute sequentially and could not respond intelligently to different inputs or situations.

Conditional Statements

Definition

Conditional statements are decision-making statements that execute different blocks of code depending on whether a specified condition evaluates to True or False.

They allow programs to make decisions and perform different actions based on different situations.

Examples:

- Checking stock availability
- Validating login credentials
- Determining grades
- Checking voting eligibility

Types of Conditional Statements

Statement	Purpose
if	Executes code when a condition is True
if-else	Executes one block if True and another block if False
if-elif-else	Checks multiple conditions
Nested if	Places one conditional statement inside another

1. if Statement

Definition

The if statement is the simplest conditional statement. It executes a block of code only when the specified condition evaluates to True. If the condition is False, the block is skipped.

Syntax

Python

```
if condition:
    statements
```

Example: Product Stock Availability

Python

```
stock = 25
```

```
if stock > 0:  
    print("Product is available in stock")
```

2. if-else Statement

Definition

The if-else statement provides two possible execution paths.

One block executes when the condition is True and another block executes when the condition is False.

Syntax

Python

```
if condition:  
    statements  
else:  
    statements
```

Example: Stock Availability

Python

```
stock = 0  
  
if stock > 0:  
    print("Product is available")  
else:  
    print("Product is out of stock")
```

3. if-elif-else Statement

Definition

The if-elif-else statement is used when multiple conditions need to be checked.

Python evaluates conditions from top to bottom.

As soon as a condition becomes True, its corresponding block executes and the remaining conditions are skipped.

Syntax

Python

```
if condition1:
    statements

elif condition2:
    statements

elif condition3:
    statements

else:
    statements
```

Example: Product Stock Levels

Python

```
stock = 5

if stock > 20:
    print("Stock is fully available")

elif stock > 0:
    print("Limited stock available")

else:
    print("Out of stock")
```

4. Nested if Statement

Definition

A Nested if is an if statement placed inside another if or else block. The inner condition is evaluated only when the outer condition is satisfied. Nested if statements are useful when multiple levels of validation are required.

Syntax

```
Python
if condition1:

    if condition2:
        statements

    else:
        statements

else:
    statements
```

Example: Product Purchase Eligibility

```
Python
stock = 10
premium_member = True

if stock > 0:

    print("Product is available")

    if premium_member:
        print("Priority delivery available")

    else:
        print("Standard delivery available")

else:
    print("Product is out of stock")
```

Comparison of Conditional Statements

Statement	Purpose	Use Case
if	Execute code when condition is True	Single condition
if-else	Handle both True and False cases	Two possible outcomes
if-elif-else	Handle multiple conditions	Multiple outcomes
Nested if	Multiple levels of checking	Complex decision-making